An Introduction to Image Compression

Speaker: Wei-Yi Wei
Advisor: Jian-Jung Ding
Digital Image and Signal Processing Lab
GICE, National Taiwan University

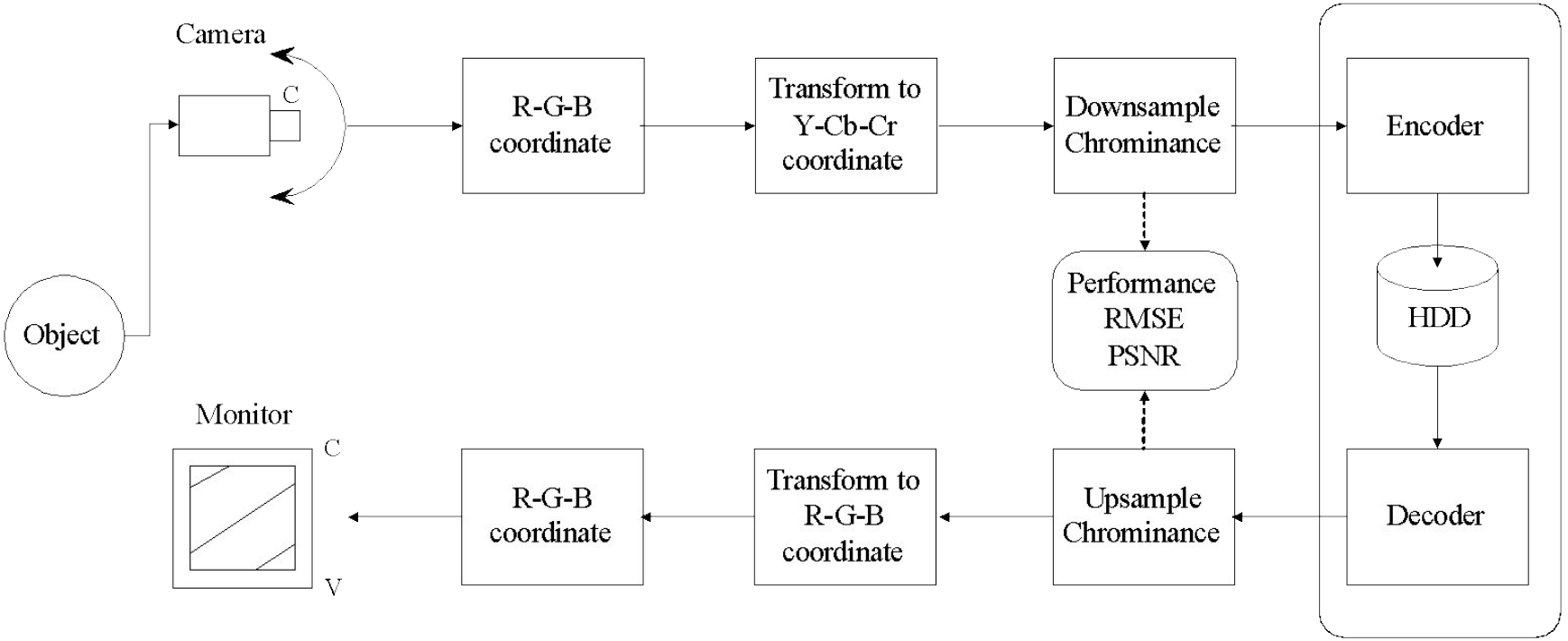
Outline

- Image Compression Fundamentals
  - General Image Storage System
  - Color Space
- Reduce correlation between pixels
  - Karhunen-Loeve Transform
  - Discrete Cosine Transform
  - Discrete Wavelet Transform
  - Differential Pulse Code Modulation
  - Differential Coding
- Quantization and Source Coding
  - Huffman Coding
  - Arithmetic Coding
  - Run Length Coding
  - Lempel Ziv 77 Algorithm
  - Lempel Ziv 78 Algorithm
- Overview of Image Compression Algorithms
  - JPEG
  - JPEG 2000
  - Shape-Adaptive Image Compression

Outline

- Image Compression Fundamentals
- Reduce correlation between pixels
- Quantization and Source Coding
- Overview of Image Compression Algorithms

General Image Storage System



Color Specification

- **Luminance**
  - Received brightness of the light, which is proportional to the total energy in the visible band.
- **Chrominance**
  - Describe the perceived color tone of a light, which depends on the wavelength composition of light
  - Chrominance is in turn characterized by two attributes
    - **Hue**
      - Specify the color tone, which depends on the peak wavelength of the light
    - **Saturation**
      - Describe how pure the color is, which depends on the spread or bandwidth of the light spectrum

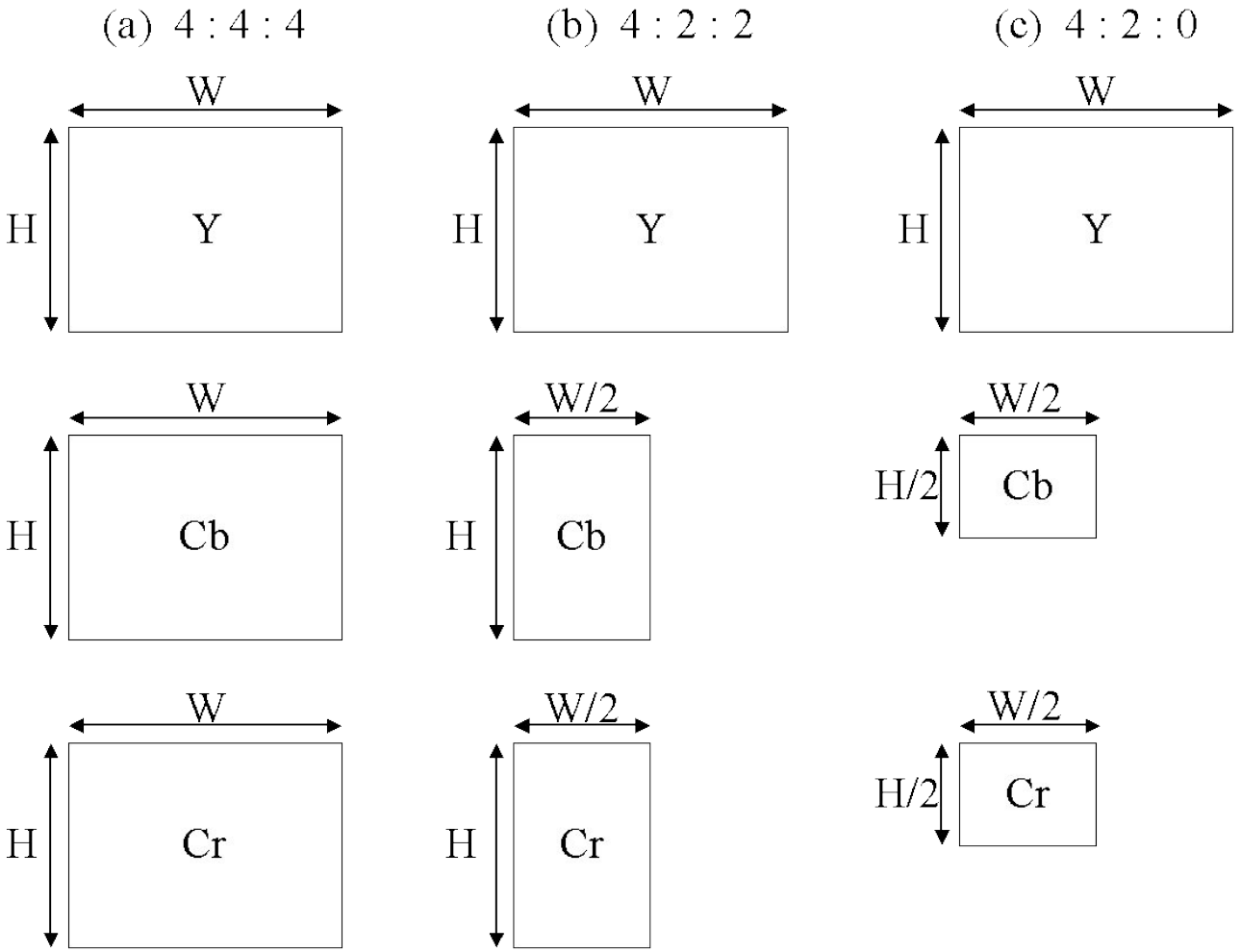
YUV Color Space

- In many applications, it is desirable to describe a color in terms of its luminance and chrominance content separately, to enable more efficient processing and transmission of color signals
- One such coordinate is the YUV color space
  - Y is the components of luminance
  - Cb and Cr are the components of chrominance
  - The values in the YUV coordinate are related to the values in the RGB coordinate by

$$\begin{pmatrix} Y \\ Cb \\ Cr \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.334 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} + \begin{pmatrix} 0 \\ 128 \\ 128 \end{pmatrix}$$

Spatial Sampling of Color Component

The three different chrominance downsampling format



The Flow of Image Compression (1/2)

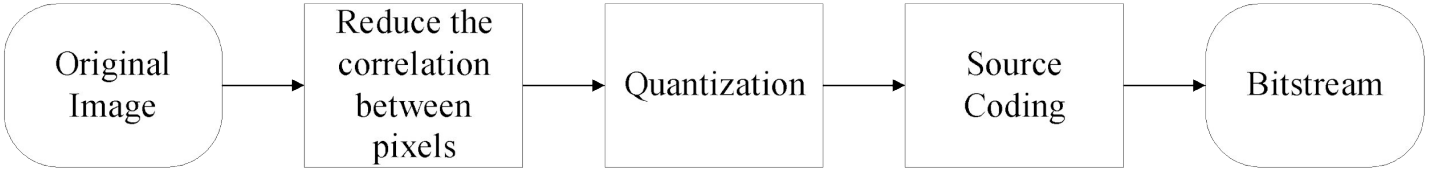
- What is the so-called image compression coding?
  - To store the image into bit-stream as compact as possible and to display the decoded image in the monitor as exact as possible
- Flow of compression
  - The image file is converted into a series of binary data, which is called the bit-stream
  - The decoder receives the encoded bit-stream and decodes it to reconstruct the image
  - The total data quantity of the bit-stream is less than the total data quantity of the original image



The Flow of Image Compression (2/2)

- Measure to evaluate the performance of image compression
  - Root Mean square error: $RMSE = \sqrt{\frac{\sum_{x=0}^{W-1}\sum_{y=0}^{H-1}[f(x,y) - f'(x,y)]^2}{WH}}$
  - Peak signal to noise ratio: $PSNR = 20\log_{10}\frac{255}{MSE}$
  - Compression Ratio: $Cr = \frac{n1}{n2}$

    Where *n1* is the data rate of original image and *n2* is that of the encoded bit-stream
- The flow of encoding
  - Reduce the correlation between pixels
  - Quantization
  - Source Coding



Outline

- Image Compression Fundamentals
- Reduce correlation between pixels
- Quantization and Source Coding
- Overview of Image Compression Algorithms

Reduce the Correlation between Pixels

- **Orthogonal Transform Coding**
  - KLT (Karhunen-Loeve Transform)
    - Maximal Decorrelation Process
  - DCT (Discrete Cosine Transform)
    - JPEG is a DCT-based image compression standard, which is a lossy coding method and may result in some loss of details and unrecoverable distortion.
- **Subband Coding**
  - DWT (Discrete Wavelet Transform)
    - To divide the spectrum of an image into the lowpass and the highpass components, DWT is a famous example.
    - JPEG 2000 is a 2-dimension DWT based image compression standard.
- **Predictive Coding**
  - DPCM
    - To remove mutual redundancy between seccessive pixels and encode only the new information

Covariance

- The covariance between two random variables X and Y, with expected value E[X]= μ_X and E[Y]= μ_Y is defined as

$$\begin{aligned}\operatorname{cov}(X,Y) &= E[(X-\mu_X)(Y-\mu_Y)] = E[XY]-\mu_X\mu_Y \\ &= \sum_X \sum_Y (X-\mu_X)(Y-\mu_Y) f(X,Y) \text{ (discrete)}\end{aligned}$$

- If entries in the column vector X = $[x_1,x_2,\ldots,x_N]^T$ are random variables, each with finite variance, then the covariance matrix C is the matrix whose (i, j) entry is the covariance

$$\mathrm{C} = E[(X-\mu_x)(X-\mu_x)^T] = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1n} \\ C_{21} & C_{22} & \cdots & C_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nn} \end{bmatrix}$$

$$\mu_x = [\mu_1 \quad \mu_2 \quad \ldots \quad \mu_n]^T = [E(x_1) \quad E(x_2) \quad \ldots \quad E(x_n)]^T$$

The Orthogonal Transform

- **The Linear Transform**
  - The forward transform $y = Ax$
  - The inverse transform $x = Vy$
  - If we want to obtain the inverse transform, we need to compute the inverse of the transform matrix since

$$V = A^{-1}$$

$$VA = A^{-1}A = I$$

$$x = Vy = V(Ax) = (A^{-1}A)x = x$$

- **The Orthogonal Transform**
  - The forward transform $y = V^T x$
  - The inverse transform $x = Vy$
  - If we want to obtain the inverse transform, we need not to compute the inverse of the transform matrix since

$$VV^T = V^T V = I$$

$$x = Vy = V(V^T x) = (V^T V)x = x$$

Transform Coding (1/2)

Original sequence $X=(x_0,x_1)$

Height	Weight
65	170
75	188
60	150
70	170
56	130
80	203
68	160
50	110
40	80

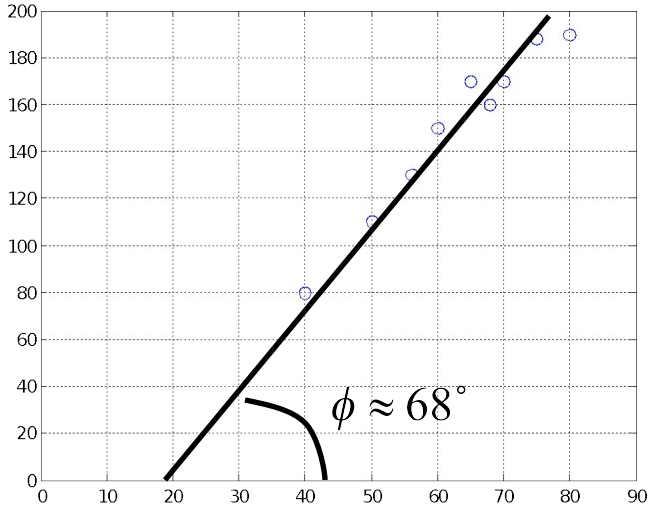
$Y = AX$

Transformed Sequence $Y=(y_0,y_1)$

Height	Weight
181.97068384838812	3.416170333863853
202.40605916474945	0.887250469682385
161.55397378997284	0.559957738379552
183.8437168154677	-1.219748938970085
141.51187032497342	-3.223438711671541
206.13345984096256	-2.999455616319722
173.822665082968	-3.111447163995635
120.72055367314222	-5.152467452589058
89.15897210197947	-7.11882670939854

The original sequence tends to cluster around the line $x_1=2.5x_0$. We rotate the sequence by the transform : $Y = AX$

$$\text{Rotation Matrix} : A = \begin{bmatrix} \cos\phi & \sin\phi \\ -\sin\phi & \cos\phi \end{bmatrix}$$



Transform Coding (2/2)

Throw the value of weight y1

Height	Weight
181.97068384838812	0
202.40605916474945	0
161.55397378997284	0
183.8437168154677	0
141.51187032497342	0
206.13345984096256	0
173.822665082968	0
120.72055367314222	0
89.15897210197947	0

$$\hat{X} = A^{-1}Y$$

Inverse Transform

Height	Weight
68.16741797800859	168.72028006870275
75.82264431044632	187.66763012404562
60.51918377426525	149.79023613916877
68.86906847716196	170.45692599485025
53.01127967035257	131.20752139486424
77.21895318005868	191.12361585053173
65.11511642520563	161.165568622698
45.222715366778566	111.93014828010075
33.399538811586865	82.66675942272599

$$\begin{bmatrix} \hat{x}0 \\ \hat{x}1 \end{bmatrix} = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix} \begin{bmatrix} y0 \\ 0 \end{bmatrix}$$

Height	Weight
65	170
75	188
60	150
70	170
56	130
80	203
68	160
50	110
40	80

Because the other element of the pair contained very little information, we could discard it without a significant effect on the fidelity of the reconstructed sequence

Karhunen-Loeve Transform

- **KLT is** the optimum transform coder that is defined as the one that minimizes the mean square distortion of the reproduced data for a given number of total bits

The KLT

X: The input vector with size N-by-1

A: The transform matrix with size N-by-N

Y: The transformed vector with size N-by-1, and each components v(k) are mutually uncorrelated

C*xixj*: The covariance matrix of *xi* and *xj*

C*yiyj*: The covariance matrix of *yi* and *yj*

The transform matrix **A** is composed of the eigenvectors of the autocorrelation matrix **C***xixj*, which makes the output autocorrelation matrix **C***yiyj* be composed of the eigenvalues $\lambda_0, \lambda_1, \ldots \lambda_{N-1}$ in the diagonal direction. That is

$$
\begin{aligned}
C_{yy} &= E[(Y - E(Y))(Y - E(Y))^T] \\
&= E[YY^T] : zero - mean\ assumption \\
&= E[(Ax)(Ax)^T] = E[(Axx^T A^T)] \\
&= AE[xx^T]A^T = AC_{xx}A^T
\end{aligned}
$$

$$
C_{yy} = \begin{bmatrix} \lambda_0 & 0 & \cdots & 0 \\ 0 & \lambda_1 & 0 & \vdots \\ \vdots & 0 & \ddots & 0 \\ 0 & \cdots & 0 & \lambda_{N-1} \end{bmatrix} = AC_{xx}A^T
$$

Discrete Cosine Transform (1/2)

- Why DCT is more appropriate for image compression than DFT?
  - The DCT can concentrate the energy of the transformed signal in low frequency, whereas the DFT can not
  - For image compression, the DCT can reduce the blocking effect than the DFT

Forward DCT

$$F(u,v) = \frac{2}{N} C(u)C(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos\left[\frac{\pi(2x+1)u}{2N}\right] \cos\left[\frac{\pi(2y+1)v}{2N}\right]$$

for $u = 0,\ldots,N-1$ and $v = 0,\ldots,N-1$

where $N = 8$ and $C(k) = \begin{cases} 1/\sqrt{2} & \text{for } k = 0 \\ 1 & \text{otherwise} \end{cases}$

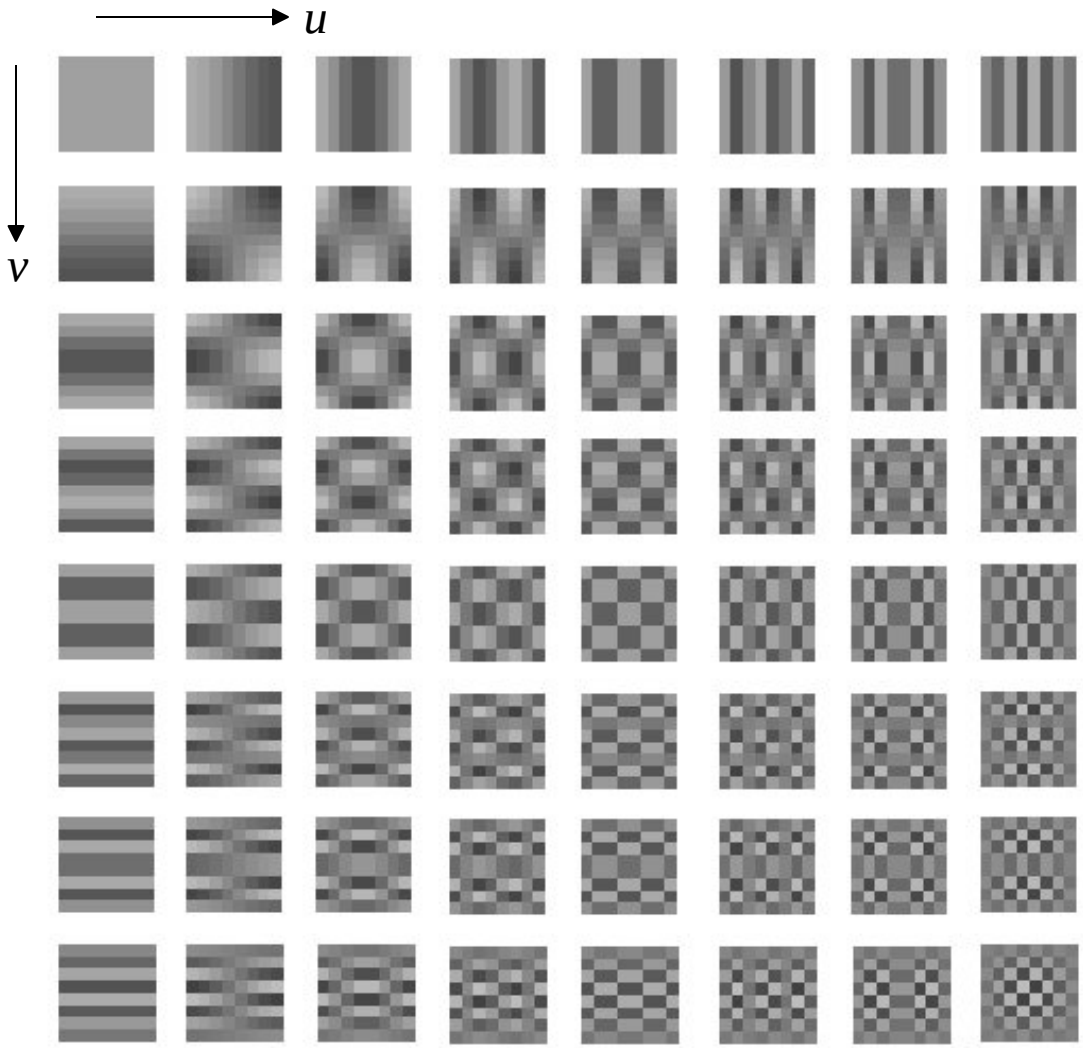
Inverse DCT

$$f(x,y) = \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} C(u)C(v)F(u,v) \cos\left[\frac{\pi(2x+1)u}{2N}\right] \cos\left[\frac{\pi(2y+1)v}{2N}\right]$$

for $x = 0,\ldots,N-1$ and $y = 0,\ldots,N-1$ where $N = 8$

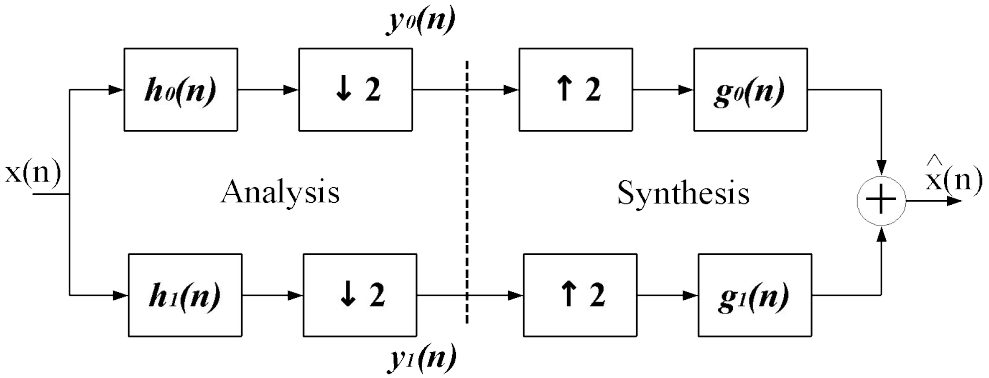
Discrete Cosine Transform (2/2)

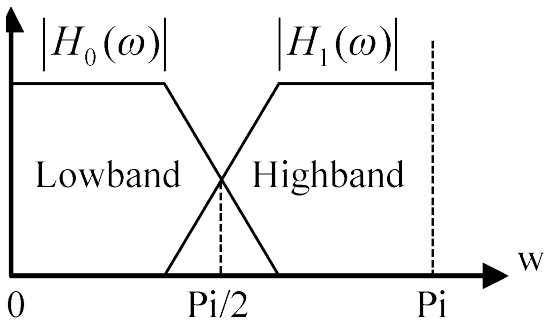
The 8-by-8 DCT basis



Discrete Wavelet Transform (1/2)

- Subband Coding
  - The spectrum of the input data is decomposed into a set of bandlimitted components, which is called subbands
  - Ideally, the subbands can be assembled back to reconstruct the original spectrum without any error
- The input signal will be filtered into lowpass and highpass components through analysis filters
- The human perception system has different sensitivity to different frequency band
  - The human eyes are less sensitive to high frequency-band color components
  - The human ears is less sensitive to the low-frequency band less than 0.01 Hz and high-frequency band larger than 20 KHz





Discrete Wavelet Transform (2/2)

- 1D DWT applied alternatively to vertical and horizontal direction line by line
- The LL band is recursively decomposed, first vertically, and then horizontally

1D scaling function $\phi(x), \phi(y)$

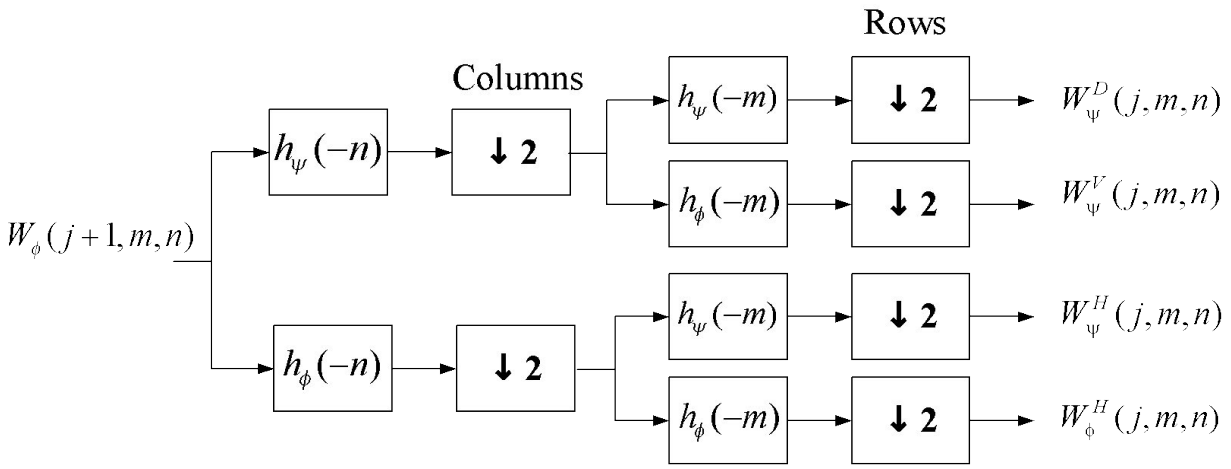
1D wavelet function $\psi(x), \psi(y)$

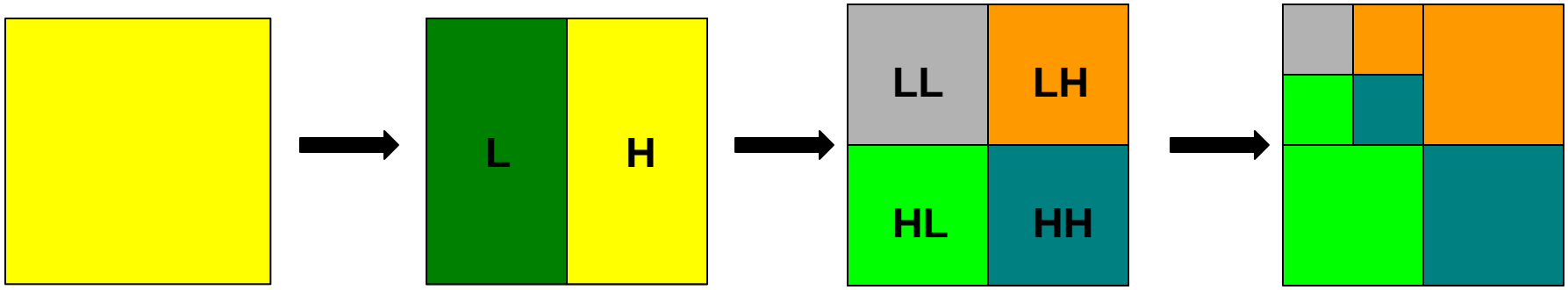
2D scaling function $\phi(x,y) = \phi(x)\phi(y)$

2D wavelet function $\psi^D(x,y) = \psi(x)\psi(y)$

$\psi^H(x,y) = \psi(x)\phi(y)$

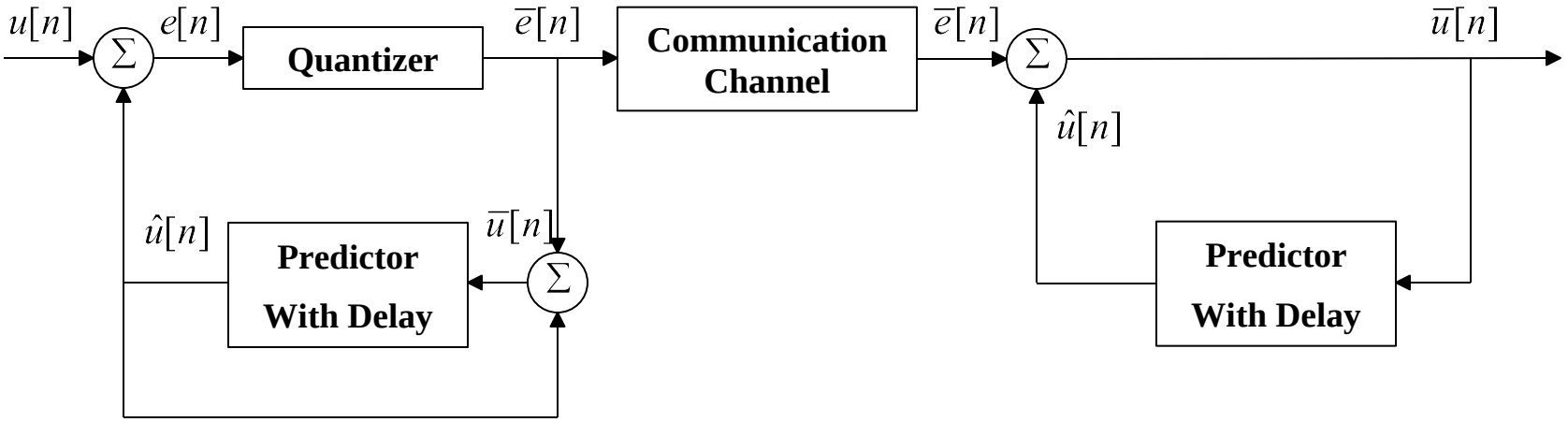
$\psi^V(x,y) = \phi(y)\psi(x)$





DPCM (1/3)

DPCM CODEC



$$u[n] = \hat{u}[u] + e[n]$$

$$u[n] - \bar{u}[u] = (\hat{u}[n] + e[u]) - (u[n] + \bar{e}[u]) = e(n) - \bar{e}[u] = q[n]$$

- There are two components to design in a DPCM system
  - The predictor
    - The predictor output $\hat{S}_0 = a_1 S_1 + a_2 S_2 + \ldots + a_n S_n$
  - The quantizer
    - A-law quantizer
    - μ-law quantizer

DPCM (2/3)

Design of Linear Predictor

$$\hat{S}_0 = a_1 S_1 + a_2 S_2 + \ldots + a_n S_n \text{ (The predictor output)},$$

$$e_0 = S_0 - \hat{S}_0 \text{ (The predictition error)}$$

$$\frac{\partial E[(S_0 - \hat{S}_0)^2]}{\partial a_i} = \frac{\partial E[(S_0 - (a_1 S_1 + a_2 S_2 + \ldots + a_n S_n))^2]}{\partial a_i}$$

$$= -2E[(S_0 - (a_1 S_1 + a_2 S_2 + \ldots + a_n S_n))S_i] = 0, \text{ for i=0, 1, 2,}\cdots\text{,n}$$

$$\Rightarrow E[(S_0 - (a_1 S_1 + a_2 S_2 + \ldots + a_n S_n))S_i] = 0$$

$$E[(S_0 - \hat{S}_0)S_i] = 0, \text{ for i=0, 1, 2,}\cdots\text{,n}$$

$$E[S_0 S_i] = E[\hat{S}_0 S_i]$$

$$R_{ij} = E[S_i S_j]$$

$$R_{0i} = E[a_1 S_1 S_i + a_2 S_2 S_i + \ldots + a_n S_n S_i] = a_1 R_{1i} + a_2 R_{2i} + \cdots + a_n R_{ni}$$

$$[R_{0i}] = [R_{1i} \quad R_{2i} \quad \cdots \quad R_{ni}] \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

$$[a_i] = [R_{1i} \quad R_{2i} \quad \cdots \quad R_{ni}]^{-1} [R_{0i}]$$

DPCM (3/3)

- When S_0 comprises these optimized coefficients, a_i, then the mean square error signal is

$$\sigma_e^2 = E[(S_0 - \hat{S}_0)^2] = E[(S_0 - S_0)S_0 - (S_0 - S_0)S_0]$$

$$= E[(S_0 - \hat{S}_0)S_0] - E[(S_0 - S_0)S_0]$$

But $E[(S_0 - \hat{S}_0)S_0] = 0$ (orthogonal principle)

$$\sigma_e^2 = E[(S_0 - \hat{S}_0)S_0] = E[(S_0)^2] - E[S_0 S_0]$$

$$= R_{00} - (a_1 R_{01} + a_2 R_{02} + \ldots + a_n R_{0n})$$

σ_e^2 : The variance of the difference signal

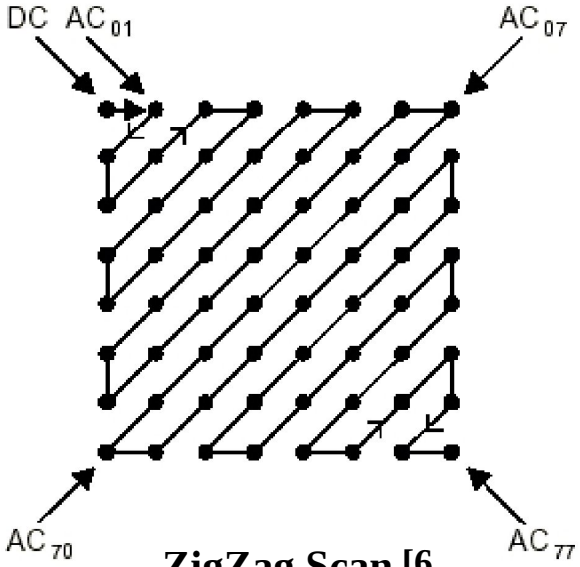
R_{00} : The variance of the original signal

⇒ The variance of the error signal is less than the variance of the original signal.

Differential Coding - JPEG (1/2)

- Transform Coefficients
  - DC coefficient
  - AC coefficients
- Because there is usually strong correlation between the DC coefficients of adjacent 8×8 blocks, the quantized DC coefficient is encoded as the difference from the DC term of the previous block
- The other 63 entries are the AC components. They are treated separately from the DC coefficients in the entropy coding process

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

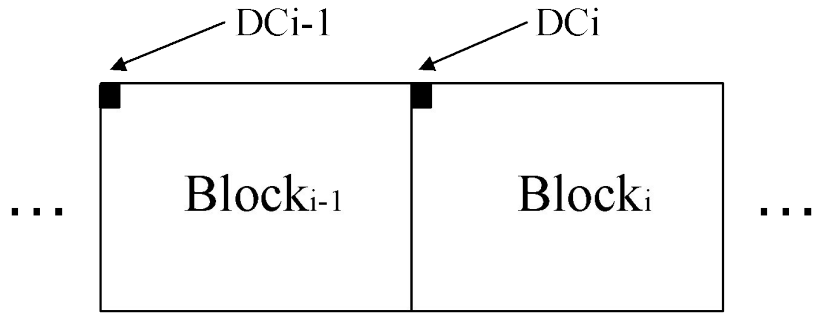


ZigZag Scan [6]

Differential Coding - JPEG (2/2)

- We set $DC_0 = 0$.
- DC of the current block DC_i will be equal to $DC_{i-1} + Diff_i$.
- Therefore, in the JPEG file, the first coefficient is actually the difference of DCs. Then the difference is encoded with Huffman coding algorithm together with the encoding of AC coefficients

Differential Coding : $Diff_i = DC_i - DC_{i-1}$



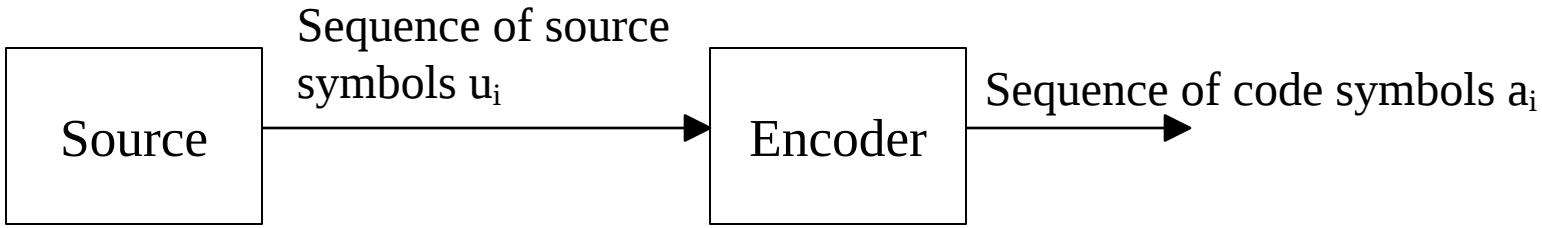
Outline

- Image Compression Fundamentals
- Reduce correlation between pixels
- Quantization and Source Coding
- Overview of Image Compression Algorithms

Quantization and Source Coding

- **Quantization**
  - The objective of quantization is to reduce the precision and to achieve higher compression ratio
  - Lossy operation, which will result in loss of precision and unrecoverable distortion
- **Source Coding**
  - To achieve less average length of bits per pixel of the image.
  - Assigns short descriptions to the more frequent outcomes and long descriptions to the less frequent outcomes
  - Entropy Coding Methods
    - Huffman Coding
    - Arithmetic Coding
    - Run Length Coding
    - Dictionary Codes
      - Lempel-Ziv77
      - Lempel-Ziv 78

Source Coding



$$U = \{u_1 \quad u_2 \quad \dots \quad u_M\}$$

$$P = \{p_1 \quad p_2 \quad \dots \quad p_M\}$$

$$A = \{a_1 \quad a_2 \quad \dots \quad a_n\}$$

meaasge $u_i \longrightarrow X_1^{(i)}, X_2^{(i)}, \cdots, X_{Ni}^{(i)}$

where $X_k^{(i)} \in A$, k=1,2,$\cdots$,N_i and i = 1,$\cdots$,M

$X^{(i)}$: codeword

N_i : length of the codeword

Average length of codeword : $\overline{N} = \sum_{i=1}^{M} p(X^{(i)})N_i = \sum_{i=1}^{M} p(u_i)N_i = \sum_{i=1}^{M} p_i N_i$

Huffman Coding (1/2)

- The code construction process has a complexity of $O(N\log_2 N)$
- Huffman codes satisfy the prefix-condition
  - Uniquely decodable: no codeword is a prefix of another codeword

Huffman Coding Algorithm

(1) Order the symbols according to the probabilities

Alphabet set: $S_1, S_2, \ldots, S_N$

Probabilities: $P_1, P_2, \ldots, P_N$

The symbols are arranged so that $P_1 \geqq P_2 \geqq \ldots \geqq P_N$

(2) Apply a contraction process to the two symbols with the smallest probabilities. Replace the last two symbols S_N and S_{N-1} to form a new symbol H_{N-1} that has the probabilities $P_1 + P_2$.

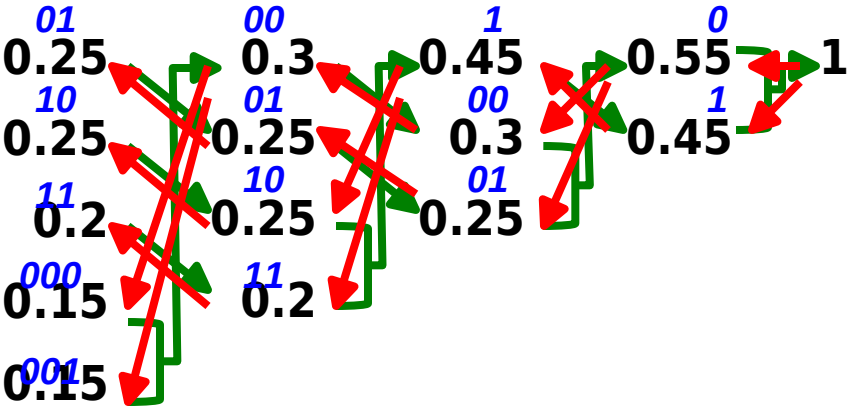
The new set of symbols has N-1 members: $S_1, S_2, \ldots, S_{N-2}, H_{N-1}$

(3) Repeat the step 2 until the final set has only one member.

(4) The codeword for each symbol S_i is obtained by traversing the binary tree from its root to the leaf node corresponding to S_i

Huffman Coding (2/2)

Codeword length	Codeword	X	Probability
2	01	1	0.25
2	10	2	0.25
3	11	3	0.2
3	000	4	0.15
3	001	5	0.15



Arithmetic Coding (1/4)

Shannon-Fano-Elias Coding

- We take **X**={1,2,…,m}, p(x)>0 for all x.
- Modified cumulative distribution function $\overline{F} = \sum_{a<x} P(a) + \frac{1}{2} P(x)$
- Assume we round off $\overline{F(x)}$ to $l(x)$, which is denoted by $\lfloor \overline{F(x)} \rfloor_{l(x)}$
- The codeword of symbol x has *l(x)* $= \left\lceil \log \frac{1}{p(x)} \right\rceil + 1$ bits
- Codeword is the binary value of $\overline{F(x)}$ with *l(x)* bits

x	*P(x)*	*F(x)*	$\overline{F(x)}$	$\overline{F(x)}$ in binary	*l(x)*	codeword
1	0.25	0.25	0.125	0.001	3	001
2	0.25	0.50	0.375	0.011	3	011
3	0.20	0.70	0.600	$0.\overline{1001}$	4	1001
4	0.15	0.85	0.775	$0.1100\overline{011}$	4	1100
5	0.15	1.00	0.925	$0.1110\overline{110}$	4	1110

Arithmetic Coding (2/4)

- **Arithmetic Coding**: a direct extension of Shannon-Fano-Elias coding calculate the probability mass function $p(x^n)$ and the cumulative distribution function $F(x^n)$ for the souece sequence x^n
  - Lossless compression technique
  - Treate multiple symbols as a single data unit

Arithmetic Coding Algorithm

Input symbol is *l*

$Previous_{low}$ is the lower bound for the old interval

$Previous_{high}$ is the upper bound for the old interval

Range is $Previous_{high}$ - $Previous_{low}$

Let $Previous_{low}$= 0, $Previous_{high}$ = 1, Range = $Previous_{high}$ – $Previous_{low}$ =1

WHILE (input symbol != EOF)

    get input symbol *l*

    Range = $Previous_{high}$ - $Previous_{low}$

    New $Previous_{low}$ = $Previous_{low}$ + Range* $interval_{low}$ of *l*

    New $Previous_{high}$ = $Previous_{low}$ + Range* $interval_{high}$ of *l*

END

Arithmetic Coding (3/4)

Symbol	Probability	Sub-interval
k	0.05	[0.00,0.05)
l	0.2	[0.05,0.25)
u	0.1	[0.20,0.35)
w	0.05	[0.35,0.40)
e	0.3	[0.40,0.70)
r	0.2	[0.70,0.90)
?	0.2	[0.90,1.00)

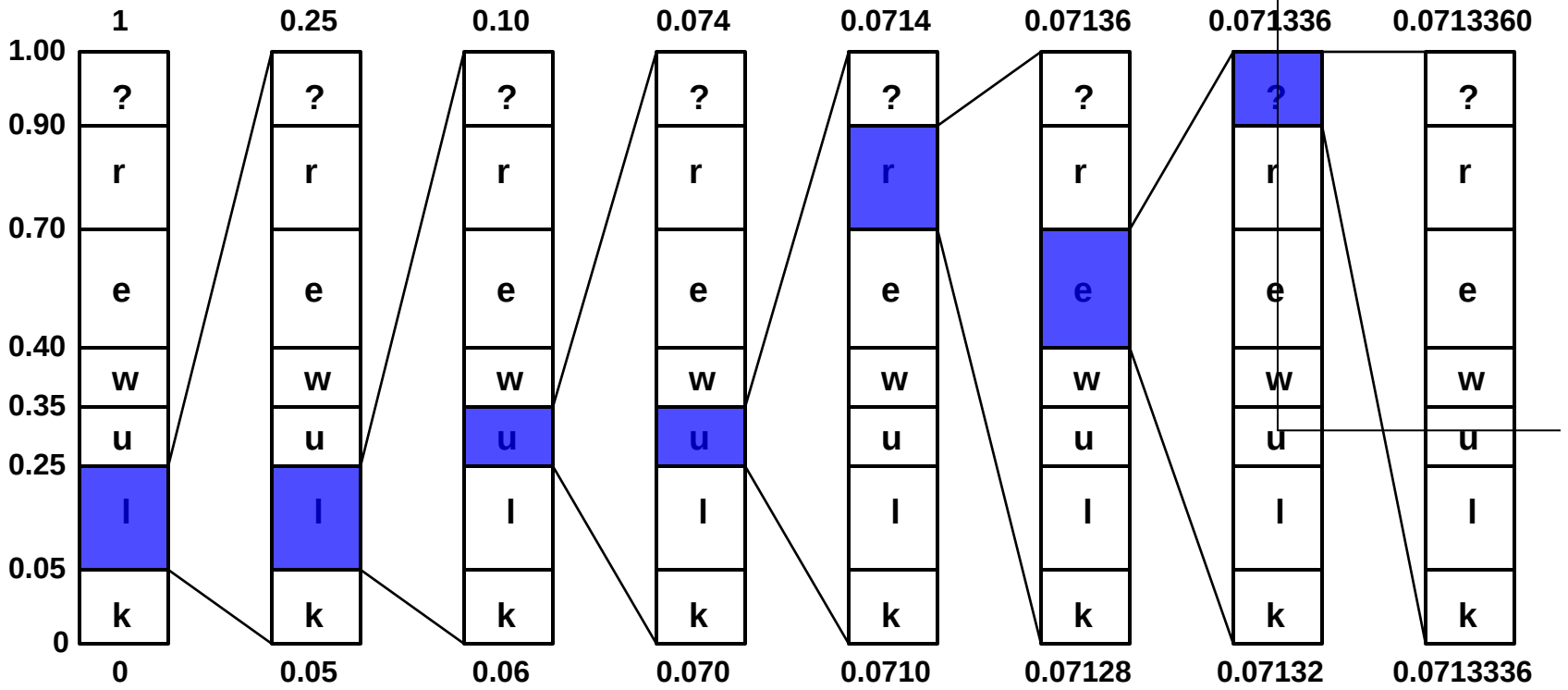
Input String : l l u u r e ?

l l u u r e ?

0.0713348389

$$=2^{-4}+2^{-7}+2^{-10}+2^{-15}+2^{-16}$$

Codeword : 0001001001000011



Arithmetic Coding (4/4)

Symbol	Probability	Huffman codeword
k	0.05	10101
l	0.2	01
u	0.1	100
w	0.05	10100
e	0.3	11
r	0.2	00
?	0.2	1011

Input String : l l u u r e ?

Huffman Coding 18 bits

Codeword : 01,01,100,100,00,11,1101

Arithmetic Coding 16 bits

Codeword : 0001001001000011

- Arithmetic coding yields better compression because it encodes a message as a whole new symbol instead of separable symbols
- Most of the computations in arithmetic coding use floating-point arithmetic. However, most hardware can only support finite precision
  - While a new symbol is coded, the precision required to present the range grows
  - There is a potential for overflow and underflow
  - If the fractional values are not scaled appropriately, the error of encoding occurs

Zero-Run-Length Coding-JPEG (1/2)

- The notation (L,F)
  - L zeros in front of the nonzero value F
- EOB (End of Block)
  - A special coded value means that the rest elements are all zeros
  - If the last element of the vector is not zero, then the EOB marker will not be added
- An Example:

1. ***57, 45, 0, 0, 0, 0, 23, 0, -30, -16, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, ..., 0***
2. ***(0,57) ; (0,45) ; (4,23) ; (1,-30) ; (0,-16) ; (2,1) ; EOB***
3. ***(0,57) ; (0,45) ; (4,23) ; (1,-30) ; (0,-16) ; (2,1) ; (0,0)***
4. ***(0,6,111001);(0,6,101101);(4,5,10111);(1,5,00001);(0,4,0111);(2,1,1);(0,0)***
5. ***1111000 1111001 , 111000 101101 , 111111110011000 10111 ,***
   11111110110 00001 , 1011 0111 , 11100 1 , 1010

Zero-Run-Length Coding-JPEG (2/2)

Huffman table of Luminance AC coefficients

run/category	code length	code word
0/0 (EOB)	4	1010
15/0 (ZRL)	11	11111111001
0/1	2	00
...	…	…
0/6	7	1111000
...	…	…
0/10	16	1111111110000011
1/1	4	1100
1/2	5	11011
...	…	…
1/10	16	1111111110001000
2/1	5	11100
...	…	…
4/5	16	1111111110011000
...	…	…
15/10	16	1111111111111110

Dictionary Codes

- Dictionary based data compression algorithms are based on the idea of substituting a repeated pattern with a shorter token
- Dictionary codes are compression codes that dynamically construct their own coding and decoding tables “on the fly” by looking at the data stream itself
- It is not necessary for us to know the symbol probabilities beforehand. These codes take advantage of the fact that, quite often, certain strings of symbols are “frequently repeated” and these strings can be assigned code words that represent the “entire string of symbols”
- Two series
  - Lempel-Ziv 77: LZ77, LZSS, LZBW
  - Lempel-Ziv 78: LZ78, LZW, LZMW

Lempel Ziv 77 Algorithm (1/4)

- **Search Buffer**: It contains a portion of the recently encoded sequence.
- **Look-Ahead Buffer**: It contains the next portion of the sequence to be encoded.
- Once the longest match has been found, the encoder encodes it with a triple <*Cp*, *Cl*, *Cs*>
  - *Cp* :the offset or position of the longest match from the lookahead buffer
  - *Cl* :the length of the longest matching string
  - *Cs* :the codeword corresponding to the symbol in the look-ahead buffer that follows the match

LZ77 Compression Algorithm

searches the search buffer for the longest match

If (longest match is found and all the characters are compared**)**

Output <*Cp*, *Cl*, *Cs*>

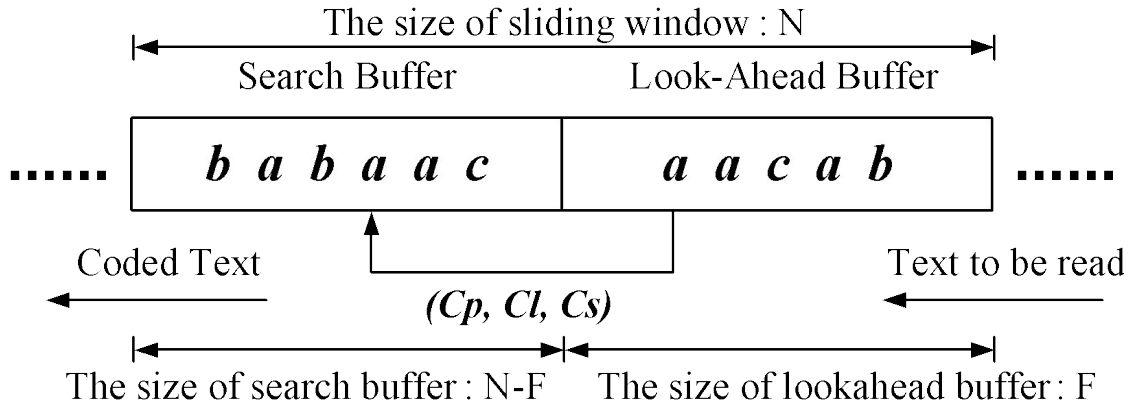
Shift window *Cl* characters

ELSE

Output <*0*, *0*, *Cs*>

Shift window 1 character

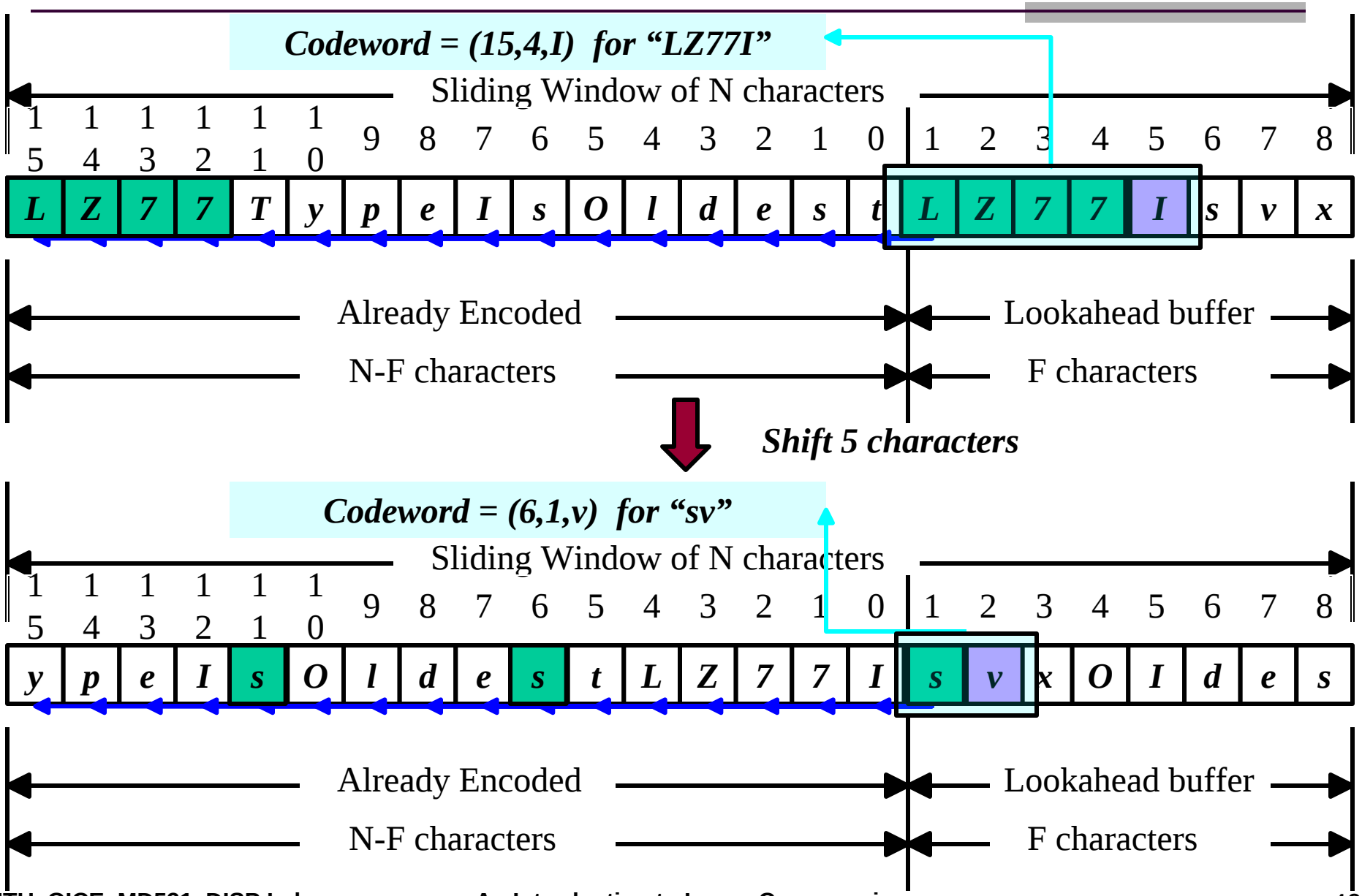
END



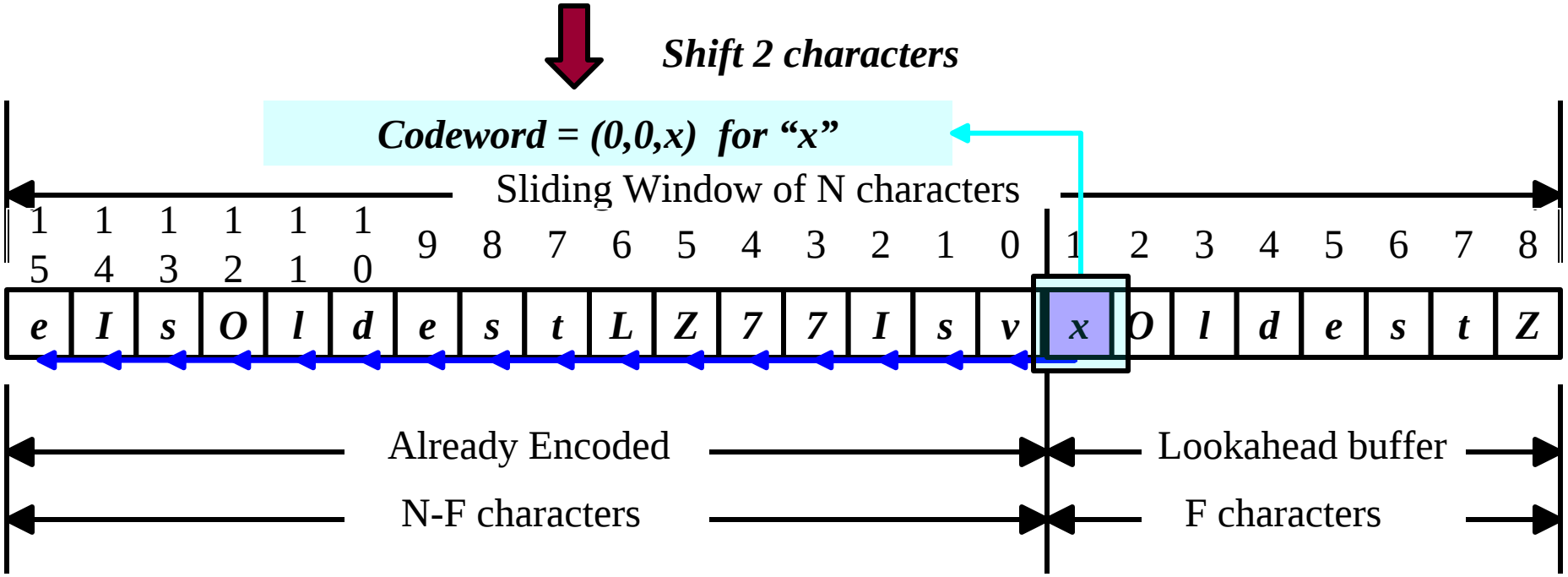
Lempel Ziv 77 Algorithm (2/4)

- Advantages of LZ77
  - Do not require to know the probabilities of the symbols beforehand
  - A particular class of dictionary codes, are known to asymptotically approach to the source entropy for long messages. That is, the longer the size of the sliding window, the better the performance of data compression
  - The receiver does not require prior knowledge of the coding table constructed by the transmitter
- Disadvantage of LZ77
  - A straightforward implementation would require up to (N-F)*F character comparisons per fragment produced. If the size of the sliding window becomes very large, then the complexity of comparison is very large

Lempel Ziv 77 Algorithm (3/4)



Lempel Ziv 77 Algorithm (4/4)



Lempel Ziv 78 Algorithm (1/3)

- The LZ78 algorithm parsed a string into phrases, where each phrase is the shortest phrase not seen so far
- The multi-character patterns are of the form: $C_0C_1 \ldots C_{n-1}C_n$. The **prefix** of a pattern consists of all the pattern characters except the last: $C_0C_1 \ldots C_{n-1}$
- This algorithm can be viewed as building a dictionary in the form of a tree, where the nodes corresponding to phrases seen so far

Lempel Ziv 78 Algorithm

- **Step 1:** In the parsing context, search the longest previously parsed phrase P matching the next encoded substring.
- **Step 2:** Identify this phrase P by its index L in a list of phrases, and place the index on the code string. Go to the innovative context.
- **Step 3:** In the innovative context, concatenate next character C to the code string, and form a new parsed phrase $P \cdot C$.
- **Step 4**: Add phrase $P \cdot C$ to the end of the list of parsed phrases as (L,C)
- **Return to the Step 1.**

Lempel Ziv 78 Algorithm (2/3)

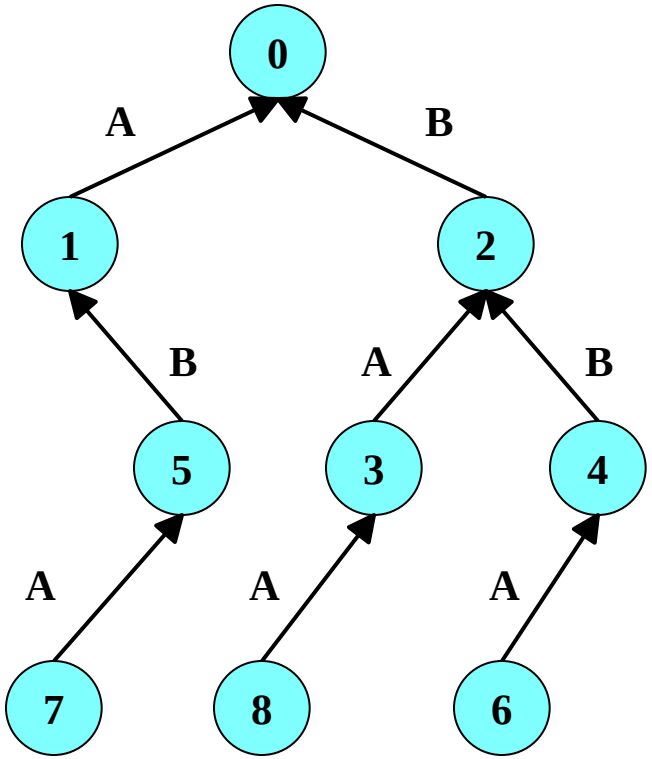
- Advantages
  - Asymptotically, the average length of the codeword per source symbol is not greater than the entropy rate of the information source
  - The encoder does not know the probabilities of the source symbol beforehand
- Disadvantage
  - If the size of the input goes to infinity, most texts are considerably shorter than the entropy of the source. However, due to the limitation of memory in modern computer, the resource of memory would be exhausted before compression become optimal. This is the bottleneck of LZ78 needs to be overcame

Lempel Ziv 78 Algorithm (3/3)

Input String: ABBABBABBBAABABAA

Parsed String: A, B, BA, BB, AB, BBA, ABA, BAA

Output Codes: (0,A), (0,B), (2,A), (2,B), (1,B), (4,A), (5,A), (3,A)



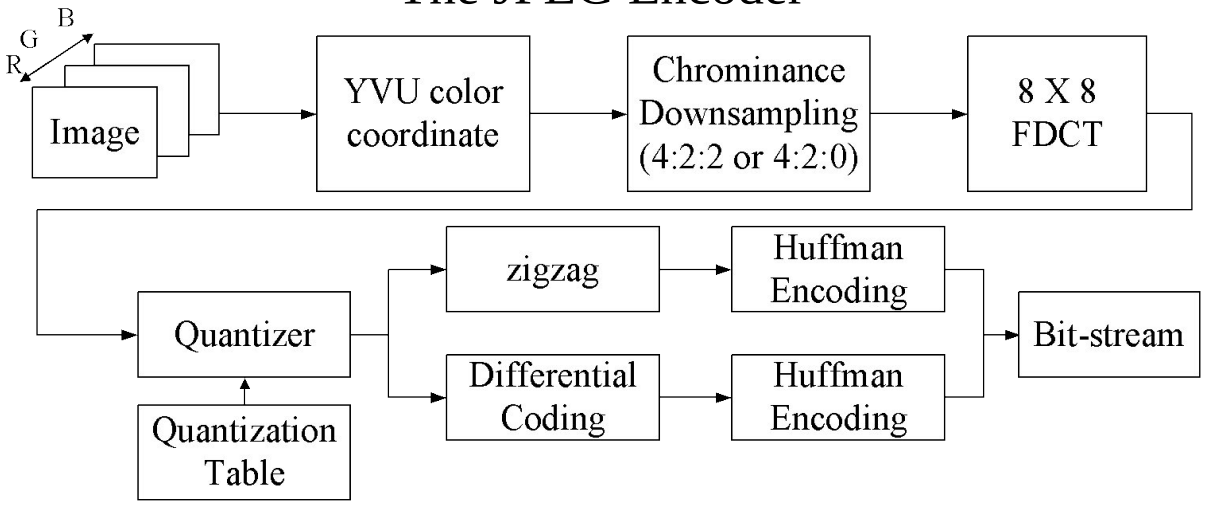
Index	Phrases	(L,C)
1 A	→	(0,A)
2 B	→	(0,B)
3 BA	→	(2,A)
4 BB	→	(2,B)
5 AB	→	(1,B)
6 BBA	→	(4,A)
7 ABA	→	(5,A)
8 BAA	→	(3,A)

Outline

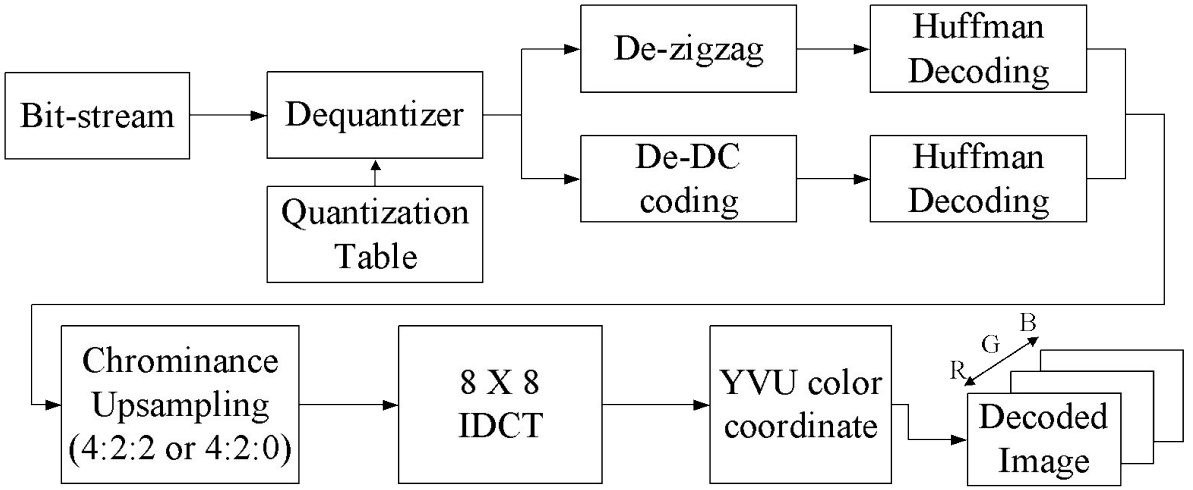
- Image Compression Fundamentals
- Reduce correlation between pixels
- Quantization and Source Coding
- Overview of Image Compression Algorithms

JPEG

The JPEG Encoder



The JPEG Decoder



Quantization in JPEG

- Quantization is the step where we actually throw away data.
- Luminance and Chrominance Quantization Table
  - lower numbers in the upper left direction
  - large numbers in the lower right direction
  - The performance is close to the optimal condition

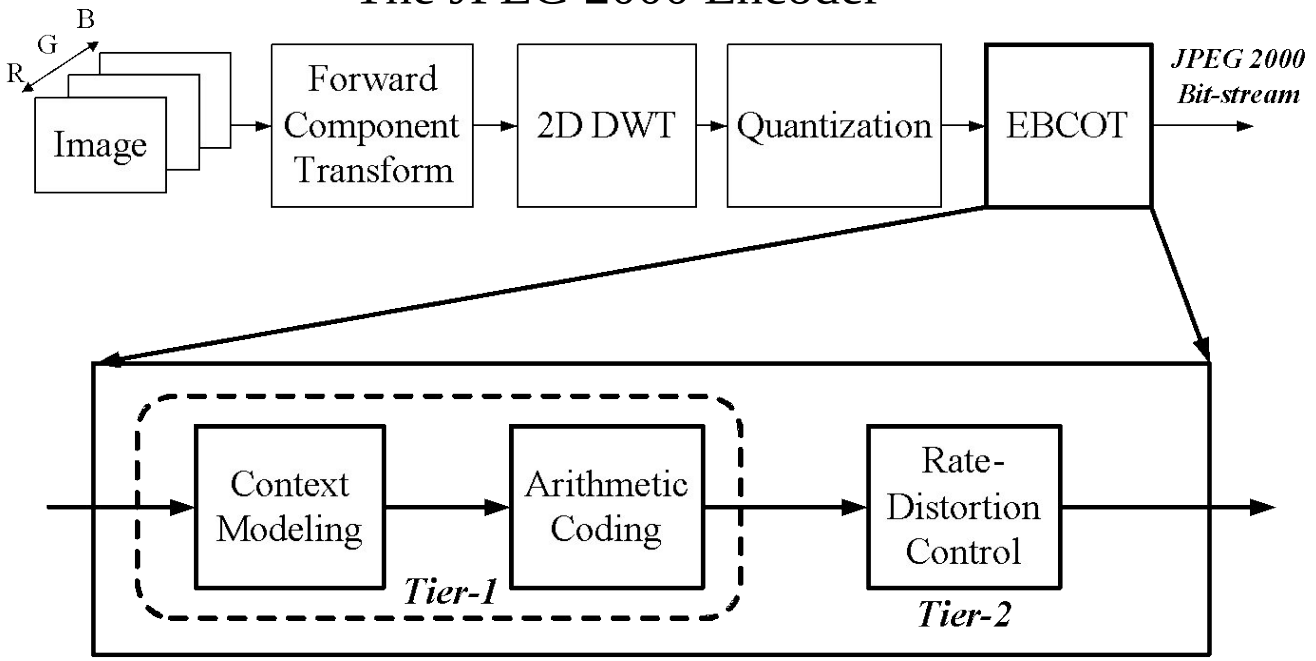
Quantization $F(u,v)_{Quantization} = round\left(\frac{F(u,v)}{Q(u,v)}\right)$

Dequantization $F(u,v)_{deQ} = F(u,v)_{Quantization} \times Q(u,v)$

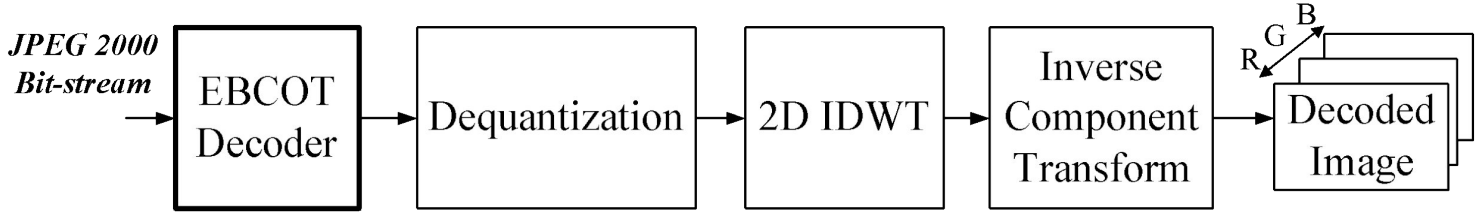
$$Q_Y = \begin{pmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{pmatrix} \quad Q_C = \begin{pmatrix} 17 & 18 & 24 & 47 & 99 & 99 & 99 & 99 \\ 18 & 21 & 26 & 66 & 99 & 99 & 99 & 99 \\ 24 & 26 & 56 & 99 & 99 & 99 & 99 & 99 \\ 47 & 66 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \end{pmatrix}$$

JPEG 2000

The JPEG 2000 Encoder



The JPEG 2000 Decoder

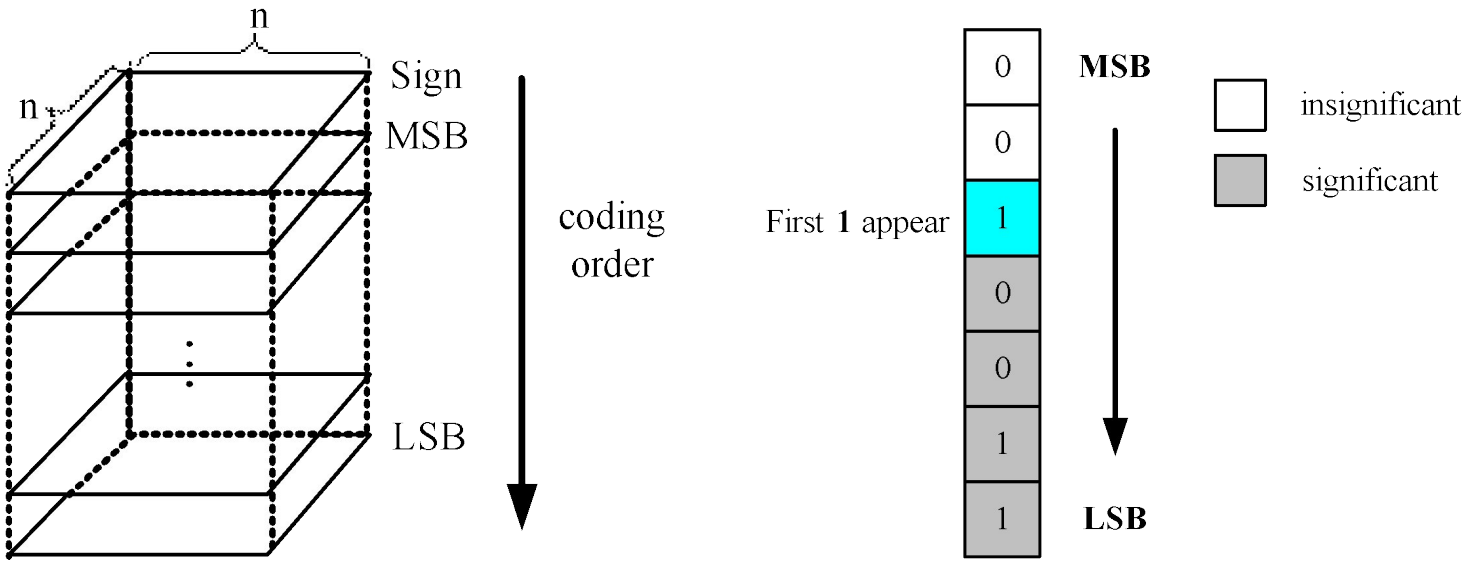


Quantization in JPEG 2000

- Quantization coefficients $q_b(u,v) = sign\left[a_u(u,v)\right] \times floor\left[\frac{|a_b(u,v)|}{\Delta_b}\right]$
  - $a_b(u,v)$: the wavelet coefficients of subband b
- Quantization step size $\Delta_b = 2^{R_b - \varepsilon_b}\left(1 + \frac{\mu_b}{2^{11}}\right)$
  - R_b: the nominal dynamic range of subband b
  - ε_b: number of bits alloted to the exponent of the subband's coefficients
  - μ_b: number of bits allotted to the mantissa of the subband's coefficients
- **Reversible wavelets**
  - Uniform deadzone scalar quantization with a step size of $\Delta b = 1$ must be used
- **Irreversible wavelets**
  - The step size is specified in terms of an exponent ε_b, $0 \leqq \varepsilon_b < 2^5$, and a mantissa μ_b , $0 \leqq \mu_b < 2^{11}$

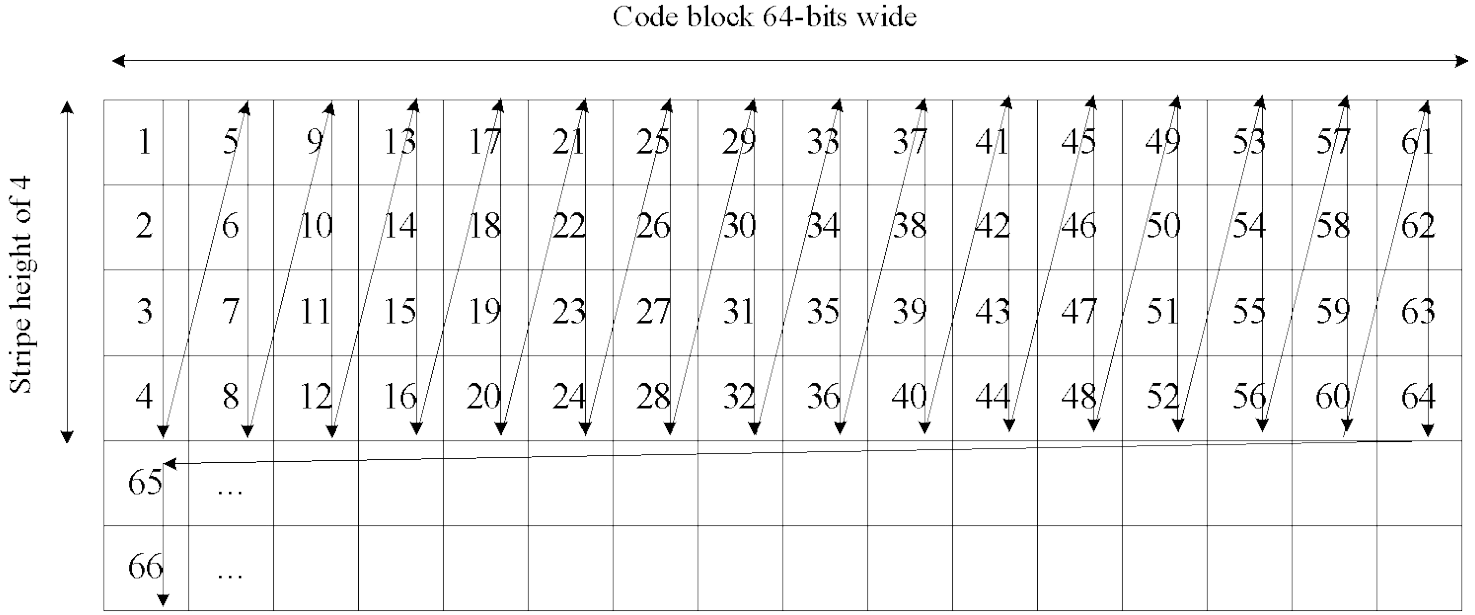
Bitplane Scanning

- The decimal DWT coefficients can be converted into signed binary format, so the DWT coefficients are decomposed into many 1-bit planes.
- In one 1-bit-plane
  - **Significant**
    - A bit is called significant after the first bit ‘1’ is met from MSB to LSB
  - **Insignificant**
    - The bits ‘0’ before the first bit ‘1’ are insignificant



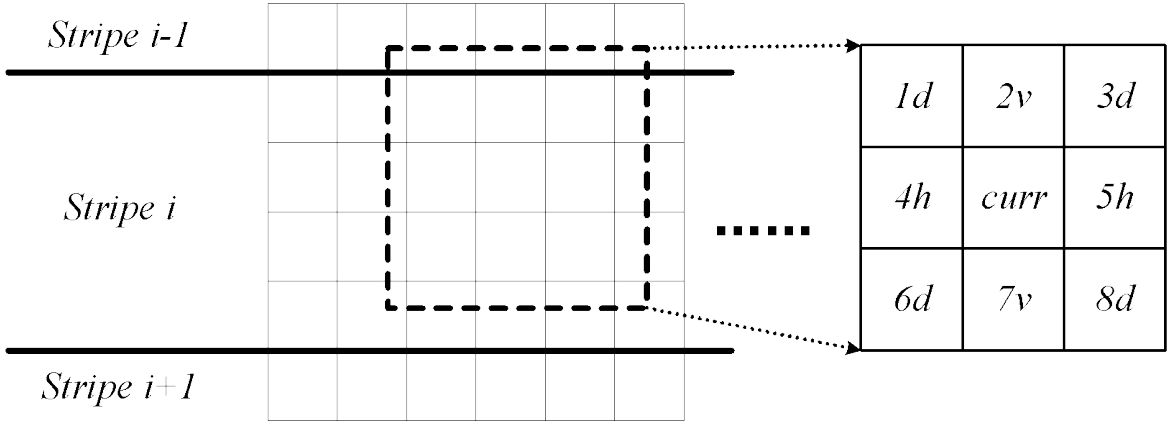
Scanning Sequence

- The scanning order of the bit-plane
- **Sample**
  - Each element of the bit-plane is called a sample
- **Column stripe**
  - Four vertical samples can form one column stripe
- **Full stripe**
  - The 16 column stripes in the horizontal direction can form a full stripe



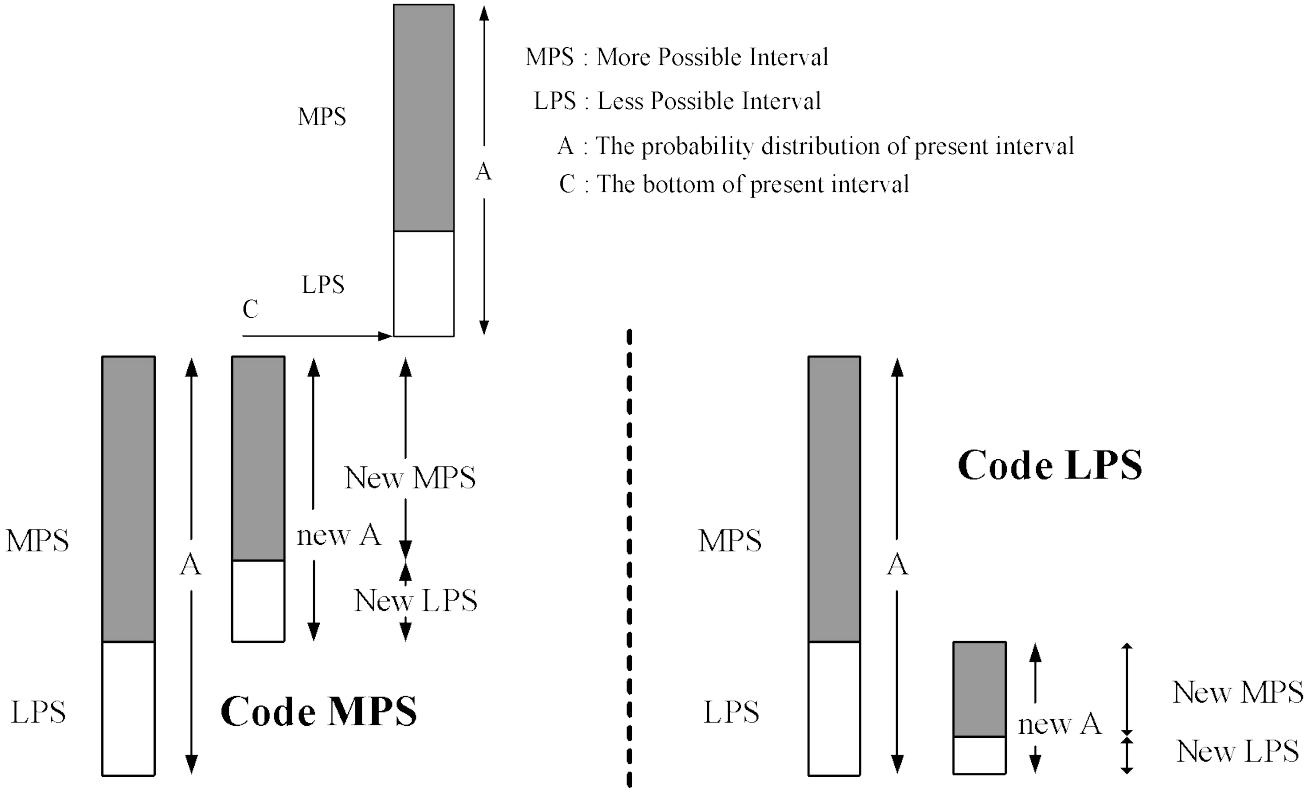
The Context Window

- **Current sample**
  - The “curr” is the sample which is to be coded
  - The other 8 samples are its neighbor samples
- **The diagonal sample**
  - The samples in the diagonal direction
- **The vertical samples**
  - The samples in the vertical direction
- **The horizontal samples**
  - The samples in the horizontal direction



Arithmetic Coder in JPEG 2000

- The decision and context data generated from context formation is coded in the arithmetic encoder
- The arithmetic encoder used by JPEG 2000 standard is a binary coder
  - More Possible Symbol (MPS): If the value of input is 1
  - Less Possible Symbol (LPS): If the value of input is 0

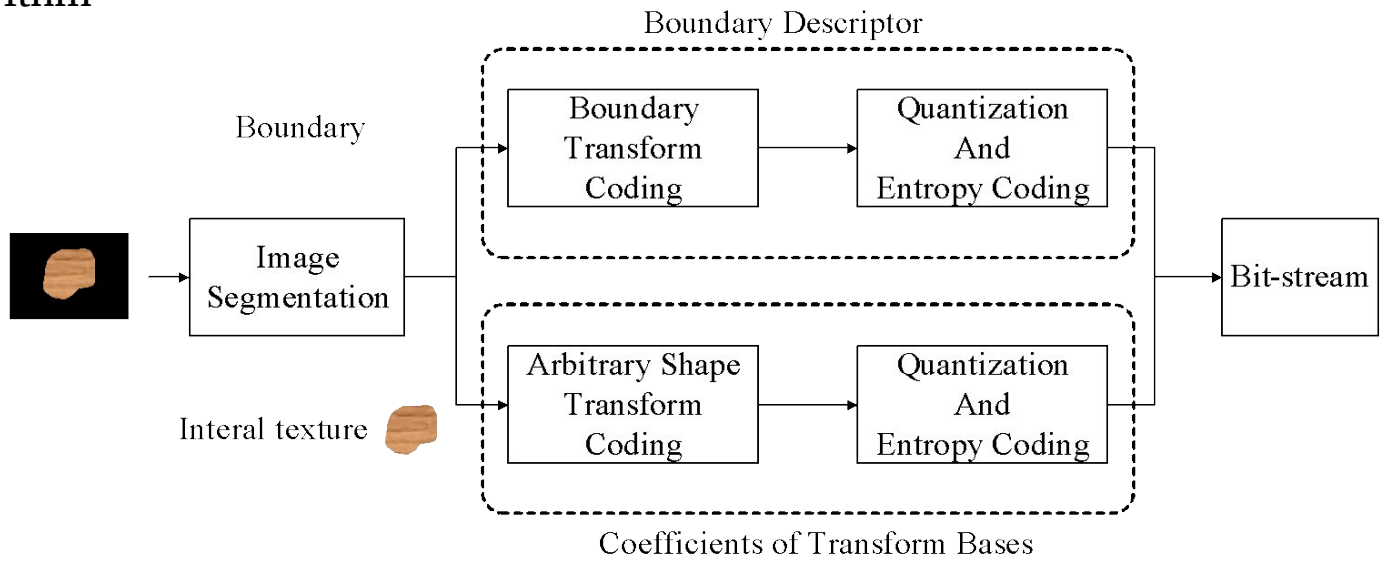


Rate Distortion Optimization

- For meeting a target bit-rate or transmission time, the packaging process imposes a particular organization of coding pass data in the output code-stream
- The rate-control assures that the desired number of bytes used by the code-stream while assuring the highest image quality possible

Shape-Adaptive Image Compression

- Both the JPEG and JPEG 2000 image compression standard can achieve great compression ratio, however, both of them do not take advantage of the local characteristics of the given image effectively
- Instead of taking the whole image as an object and utilizing transform coding, quantization, and entropy coding to encode this object, the SAIC algorithm segments the whole image into several objects, and each object has its own local characteristic and color
- Because of the high correlation of the color values in each image segment, the SAIC can achieve better compression ratio and quality than conventional image compression algorithm



Reference

- [1] R. C. Gonzalea and R. E. Woods, "*Digital Image Processing*", 2nd Ed., Prentice Hall, 2004.
- [2] Liu Chien-Chih, Hang Hsueh-Ming, "Acceleration and Implementation of JPEG 2000 Encoder on TI DSP platform" *Image Processing, 2007.* ICIP 2007. IEEE International Conference on, Vo1. 3, pp. III-329-339, 2005.
- [3] ISO/IEC 15444-1:2000(E), "Information technology-JPEG 2000 image coding system-Part 1: Core coding system", 2000.
- [4] Jian-Jiun Ding and Jiun-De Huang, "Image Compression by Segmentation and Boundary Description", Master's Thesis, National Taiwan University, Taipei, 2007.
- [5] Jian-Jiun Ding and Tzu-Heng Lee, "Shape-Adaptive Image Compression", Master's Thesis, National Taiwan University, Taipei, 2008.
- [6] G. K. Wallace, "*The JPEG Still Picture Compression Standard*", *Communications of the ACM*, Vol. 34, Issue 4, pp.30-44, 1991.
- [7] 張得浩，"*新一代JPEG 2000之核心編碼 — 演算法及其架構(上)*"，IC設計月刊　2003.8月號.
- [8] 酒井善則、吉田俊之 共著，白執善 編譯，"*影像壓縮技術*"，全華，2004.
- [9] Subhasis Saha, "Image Compression - from DCT to Wavelets : A Review", available in http://www.acm.org/crossroads/xrds6-3/sahaimgcoding.html

Thank You

Q & A

Question 1

- In Slide DPCM(3), what's the orthogonal principle?

Orthogonal principle : $E[(S_0 - \hat{S}_0)S_0] = 0$

Proof:

Given $E[(S_0 - \hat{S}_0)S_i] = 0$

$$E[(S_0 - \hat{S}_0)S_0]$$
$$= E[(S_0 - \hat{S}_0)(a_1S_1 + a_2S_2 + \ldots + a_nS_n)]$$
$$= a_1E[(S_0 - \hat{S}_0)S_1] + a_2E[(S_0 - S_0)S_2] + \cdots a_nE[(S_0 - S_0)S_n]$$
$$= 0 + 0 + \cdots + 0$$
$$= 0$$